Model fitting report: SEIQR model for the 2026 Bundibugyo outbreak, DRC

Team TAUS | companion to the manuscript | every number below is generated by `R/02_fit.R` and `R/07_figures_tables.R`

1 What is fitted and why

The scenarios need three data-determined inputs: the current growth rate, the current incidence, and the date transmission changed. Case counts cannot identify the natural history, the baseline delay or the isolation effectiveness ϵ; these are fixed (latent period 6.3 d, infectious period 7 d, baseline delay 7 d, $\epsilon = 0.8$) and varied afterwards. The fitted quantities are the starting incidence i_0 (on 2 May 2026), transmission rates β_1, β_2 before and after a change date t_c, and the negative-binomial dispersion k (counting t_c, five parameters, against three for constant transmission).

2 Data

Fit B (primary): confirmed cases by report date, 2 June–12 September (96 report intervals) from the BVDOutbreakSize streams [1]. The 22 July harmonisation step (369 reported, 97 new) is replaced by the printed 97, removing 272 retrospective cases. Fit A (cross-check): confirmed cases by onset date from SitRep 121 (25 May–6 September, 105 days; about 66% of confirmed cases have a recorded onset), corrected for right-truncation. Suspected cases, bed capacity and treatment-centre streams stop or change definition and are not used.

3 Reporting-delay correction

Each SitRep republishes the onset curve, so the share of an onset day's final count visible L days later can be measured over 55 vintages (final = counts at least 28 days old in the latest vintage). A Weibull cumulative distribution $c(L) = 1 - \exp\{-((L + 0.5)/s)^\kappa\}$ fits it: half of cases visible after 7 days, 90% after 15 days, mean 8.5 days. This explains the apparent collapse of the onset curve in the last three weeks (Fig. 1a,b).

4 Model, likelihood and estimation

Model (1) is the SEIQR system with $\lambda = \beta[I + (1 - \epsilon)Q]/N$:

$$\dot{S} = -\lambda S,\ \dot{E} = \lambda S - \sigma E,\ \dot{I} = \sigma E - (\tau + \gamma)I,\ \dot{Q} = \tau I - \gamma Q,\ \dot{R} = \gamma(I + Q). \tag{1}$$

Symptomatic incidence is $i(d) = \sigma E(d)$. Observations are negative binomial with mean μ and variance $\mu + \mu^2/k$. In Fit A, $\mu_d = c(L_d)\, i(d)$. In Fit B the expected count over a report interval is the incidence convolved with the daily delay probabilities $c(L) - c(L - 1)$ and summed over the interval's days. The transmission rate changes from β_1 to β_2 on day t_c. Maximum likelihood used Nelder–Mead from several starts; t_c was profiled on a 5-day grid and refined to 1 day. Uncertainty draws sample t_c with weights proportional to the profile likelihood and the other parameters from the Hessian-based multivariate normal at that t_c. ΔAIC compares with constant transmission.

5 Results

Table 1: Fitted parameters (median and 95% uncertainty interval), $\epsilon = 0.8$, baseline delay 7 d; growth rates per day.

Fit	Growth rate before (/d)	Growth rate after (/d)	R_e before	R_e after	Change date (95% interval)	ΔAIC	Incidence 12 Sep (/d)
A: onset curve	0.0169 (0.0112–0.0234)	0.0040 (−0.0015 to 0.0065)	1.19 (1.13–1.27)	1.04 (0.98–1.07)	5 July (28 June–2 August)	14.5	63 (55–71)
B: report date	0.0177 (0.0140–0.0216)	0.0004 (−0.0062 to 0.0058)	1.20 (1.16–1.25)	1.00 (0.93–1.06)	24 July (18 July–7 August)	15.4	83 (68–100)

A change in transmission was strongly supported (ΔAIC 14.5 for Fit A, 15.4 for Fit B). In Fit B growth fell from 0.018 (0.014 to 0.022) to 0.000 (−0.006 to 0.006) per day around 24 July (interval 18 July to 7 August), i.e. R_e from 1.20 (1.16 to 1.25) to 1.00 (0.93 to 1.06). The model incidence on 12 September was 83 (68 to 100) per day (Fit A: 63 (55 to 71), covering only cases with an onset date).

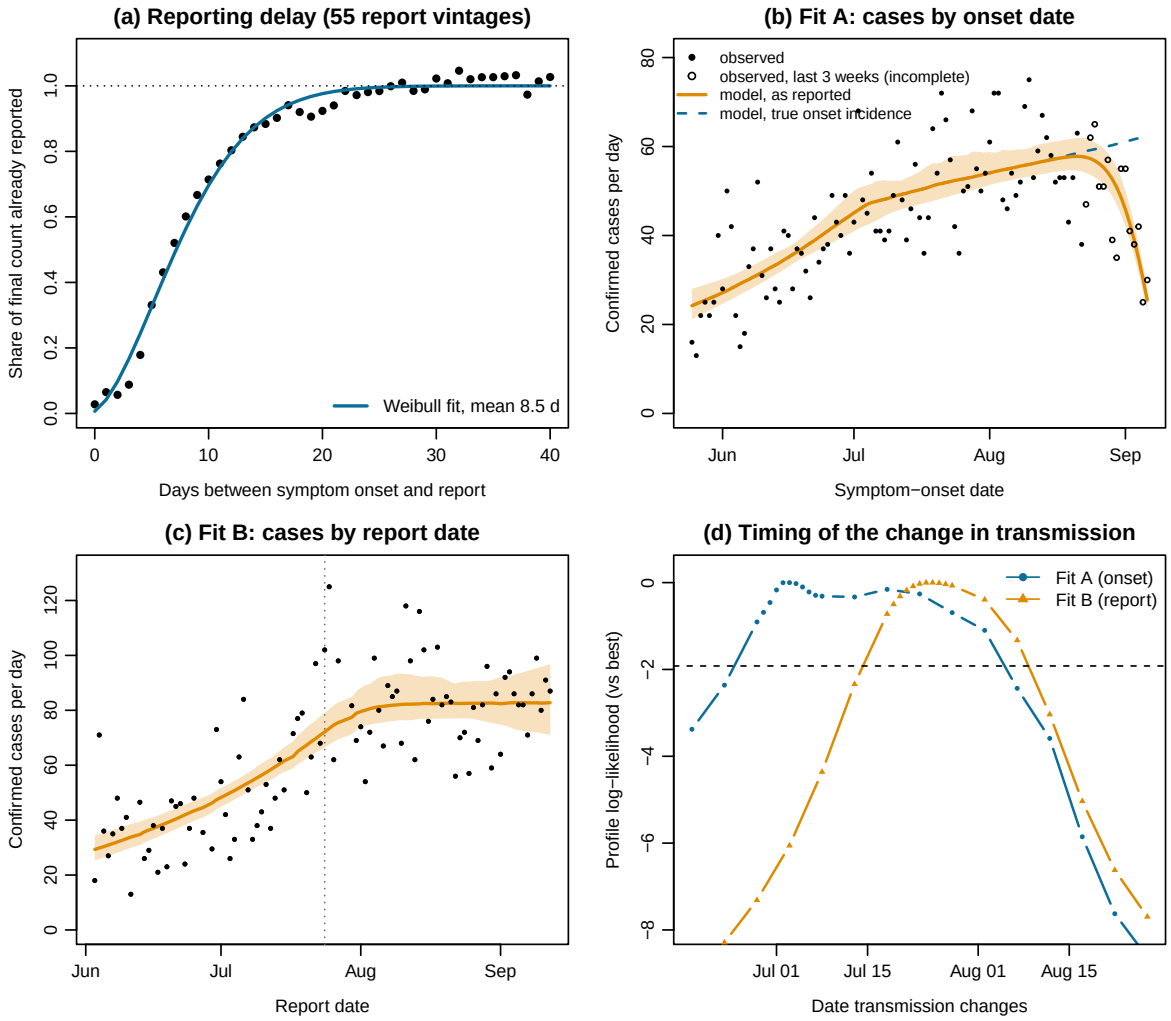


Figure 1: Fitting outcome. (a) Reporting-delay curve from 55 vintages. (b) Fit A with right-truncation correction. (c) Fit B, report-date counts with the change date. (d) Profile log-likelihood for the change date.

6 Diagnostics

95% negative-binomial predictive intervals covered 96% (Fit A) and 95% (Fit B) of observations; lag-1 residual autocorrelation was 0.27 and 0.01. Refitting Fit B at the four corners of the natural-history ranges changed the post-change growth rate only between −0.0005 and 0.0009 per day and the 12 September incidence between 81 and 84 (Table 2), so these two quantities are determined by the data rather than by the assumed natural history. Because β and ϵ are not separately identifiable, the growth rates are identical for every ϵ (Table 3). Testing expanded from 168 to 494 samples per day while positivity fell from 22.6% to 17.1% (Table 4), so the plateau is not simply a laboratory-capacity ceiling.

Table 2: Fit B refitted at extreme natural-history values.

Latent (d)	Infectious (d)	Neg. log-lik.	β after	Growth rate after (/d)	Incidence 12 Sep (/d)
4.0	5	407.60	0.300	-0.0001	81.7
10.1	5	407.59	0.298	-0.0005	81.2
4.0	10	407.62	0.191	0.0009	83.5
10.1	10	407.58	0.190	0.0002	82.4

Table 3: Fit by isolation effectiveness.

Fit	Isolation eff.	β before	β after	Growth after (/d)	R_e before	R_e after
A	50%	0.235	0.201	0.0046	1.24	1.06
B	50%	0.233	0.191	0.0003	1.23	1.00
A	80%	0.288	0.250	0.0044	1.21	1.05
B	80%	0.286	0.238	0.0000	1.20	1.00
A	100%	0.339	0.298	0.0043	1.19	1.04
B	100%	0.337	0.285	-0.0003	1.18	1.00

Table 4: Testing and positivity by month.

Month (2026)	New confirmed cases	Samples analysed	Samples per day	Positivity (%)
Jun	987	4,365	168	22.6
Jul	1,996	9,102	294	21.9
Aug	2,512	13,373	431	18.8
Sep	1,014	5,927	494	17.1

7 Out-of-sample check

A forecast using data to 12 September overestimated the next 46 days by 23% (model 3,781 (2,796 to 4,991) against 3,074 observed; observed inside the interval), and the next 15 days by 35% (outside). A constant-rate benchmark erred similarly (24% at 46 days).

8 Reproduce

`Rscript R/run_all.R` (about 15 min) runs tests, `01_data_prep.R`, `02_fit.R` and the remaining analysis, and `Rscript R/08_build_reports.R` rebuilds this report. Code for the fitting steps follows.

A Code: data preparation (R/01_data_prep.R)

```r
## 01_data_prep.R -- load the surveillance data, measure the reporting delay, build the two fitting datasets.
## Inputs : data/bvd_2026/*.csv  (epiforecasts/BVDOutbreakSize export, data as of 12 Sep 2026)
## Outputs: results/prep.rds, results/tables/data_summary.csv, results/positivity.csv
find_root_ <- function() { a <- commandArgs(FALSE); f <- sub("--file=", "", a[grep("--file=", a)]); normalizePath(file.path(dirname(f[1]),
      ".."), mustWork = FALSE) }
source(file.path(find_root_(), "R", "00_functions.R"))
set.seed(20261101)

rd <- function(f, ...) read.csv(file.path(P$bvd, f), stringsAsFactors = FALSE, ...)
wide <- rd("bvd_2026_streams_wide.csv"); wide$date <- as.Date(wide$date)
lat  <- rd("bvd_2026_onset_curve_latest.csv"); lat$onset_date <- as.Date(lat$onset_date); REPORT <- as.Date(lat$report_date[1])
vint <- rd("bvd_2026_onset_curve_all_vintages.csv", colClasses = c(sitrep = "character"))
vint$onset_date <- as.Date(vint$onset_date); vint$report_date <- as.Date(vint$report_date)
sc <- rd("bvd_2026_scalars.csv")
cb <- sc[sc$name == "confirmed_break_dates", ]
break_date  <- as.Date(gsub("[^0-9-]", "", cb$value))
break_gross <- as.numeric(sub(".*'gross_cases': \\[(\\d+)\\].*", "\\1", cb$other_fields))
stopifnot(REPORT == CFG$cutoff, length(break_date) == 1, is.finite(break_gross))

## ---- 1. reporting delay (completeness of an onset day's count after L days), from all report vintages
ref <- vint[vint$sitrep == "121" & vint$onset_date <= as.Date("2026-08-15"), c("onset_date", "confirmed_total")]
mm <- merge(vint, ref, by = "onset_date", suffixes = c("", "_ref"))
mm$lag <- as.integer(mm$report_date - mm$onset_date); mm <- mm[mm$lag >= 0 & mm$lag <= 40, ]
agg <- aggregate(cbind(confirmed_total, confirmed_total_ref) ~ lag, mm, sum)
agg$ratio <- agg$confirmed_total / agg$confirmed_total_ref
wfit <- optim(c(1.5, 10), function(p) if (any(p <= 0)) 1e9 else sum((agg$ratio - pweibull(agg$lag + 0.5, p[1], p[2]))^2))
WK <- wfit$par[1]; WS <- wfit$par[2]
delay <- list(shape = WK, scale = WS, mean = WS * gamma(1 + 1 / WK),
              q50 = qweibull(.5, WK, WS) - .5, q90 = qweibull(.9, WK, WS) - .5, n_vintages = length(unique(vint$sitrep)),
              rmse = sqrt(mean((agg$ratio - pweibull(agg$lag + 0.5, WK, WS))^2)))

## ---- 2. Fit A data: confirmed cases by symptom-onset date (latest vintage), 25 May -- 6 Sep
dA <- lat[lat$onset_date >= as.Date("2026-05-25"), c("onset_date", "confirmed_total")]
dA$lag <- as.numeric(REPORT - dA$onset_date); dA$k <- as.numeric(dA$onset_date - CFG$t0)

## ---- 3. Fit B data: confirmed increments between report dates, 2 Jun -- 12 Sep, harmonisation step removed
cum <- wide[!is.na(wide$confirmed_case_history) & wide$date >= as.Date("2026-06-02"), c("date", "confirmed_case_history")]
dB <- data.frame(date = cum$date[-1], prev = cum$date[-nrow(cum)], y = diff(cum$confirmed_case_history))
dB$ndays <- as.numeric(dB$date - dB$prev)
step <- dB$y[dB$date == break_date]; retro <- step - break_gross; dB$y[dB$date == break_date] <- break_gross
dB$u <- as.numeric(dB$date - CFG$t0); dB$u0 <- as.numeric(dB$prev - CFG$t0)
tst <- wide[!is.na(wide$tests_analysed_daily_history), c("date", "tests_analysed_daily_history")]
dB$tests <- sapply(seq_len(nrow(dB)), function(j) {
  d <- tst$tests_analysed_daily_history[tst$date > dB$prev[j] & tst$date <= dB$date[j]]
  if (length(d)) mean(d) * dB$ndays[j] else NA })
dB$month <- format(dB$date, "%b")

## ---- 4. positivity diagnostic (new confirmed / samples analysed)
pos <- aggregate(cbind(y, tests) ~ month, dB[!is.na(dB$tests), ], sum)
pos$days <- aggregate(ndays ~ month, dB[!is.na(dB$tests), ], sum)$ndays
pos <- pos[match(c("Jun", "Jul", "Aug", "Sep"), pos$month), ]
pos$positivity <- 100 * pos$y / pos$tests; pos$tests_per_day <- pos$tests / pos$days
names(pos)[2:3] <- c("new_confirmed", "tests_analysed")
write.csv(pos, file.path(P$res, "positivity.csv"), row.names = FALSE)

## ---- 5. descriptive summary
last <- wide[wide$date == CFG$cutoff, ]
desc <- data.frame(
  quantity = c("confirmed cases (cumulative, 12 Sep)", "confirmed deaths (cumulative, 12 Sep)", "crude deaths/cases",
               "onset-curve cases (latest vintage)", "onset-curve share of confirmed", "report intervals in Fit B",
               "retrospective cases removed (22 Jul)", "mean new confirmed per day, 1-12 Sep"),
  value = c(last$confirmed_case_history, last$confirmed_death_history, last$confirmed_death_history / last$confirmed_case_history,
            sum(lat$confirmed_total), sum(lat$confirmed_total) / last$confirmed_case_history, nrow(dB), retro,
            (last$confirmed_case_history - wide$confirmed_case_history[wide$date == as.Date("2026-09-01")]) / 11))
write.csv(desc, file.path(P$tab, "data_summary.csv"), row.names = FALSE)

saveRDS(list(agg = agg, delay = delay, dA = dA, dB = dB, pos = pos, REPORT = REPORT, break_date = break_date,
             break_gross = break_gross, retro = retro, lat = lat, last = last, desc = desc), file.path(P$res, "prep.rds"))
cat(sprintf("Reporting delay: Weibull shape %.2f scale %.1f d, mean %.1f d, 50%% by %.0f d, 90%% by %.0f d (%d vintages)\n",
            WK, WS, delay$mean, delay$q50, delay$q90, delay$n_vintages))
cat(sprintf("Fit A: %d onset days | Fit B: %d report intervals | %d retrospective cases removed\n", nrow(dA), nrow(dB), retro))
print(desc); print(pos)
```

B Code: fitting (R/02_fit.R)

```r
## 02_fit.R -- fit the SEIQR model to the 2026 DRC surveillance data (maximum likelihood, negative-binomial counts).
## Natural-history parameters are fixed at central literature values; estimated: starting incidence, transmission rate
## (optionally changing once on an estimated date) and count dispersion. Uncertainty: profile likelihood over the change
## date + multivariate-normal (Hessian) approximation for the other parameters.
## Outputs: results/fit.rds, results/fit_draws_{A,B}.csv, results/tables/fit_*.csv
find_root_ <- function() { a <- commandArgs(FALSE); f <- sub("--file=", "", a[grep("--file=", a)]); normalizePath(file.path(dirname(f[1]),
      ".."), mustWork = FALSE) }
source(file.path(find_root_(), "R", "00_functions.R"))
set.seed(20261101); t_start <- Sys.time()
pr <- readRDS(file.path(P$res, "prep.rds")); dA <- pr$dA; dB <- pr$dB
TEND <- as.numeric(CFG$cutoff - CFG$t0); WK <- pr$delay$shape; WS <- pr$delay$scale
cc <- function(L) pweibull(L + 0.5, WK, WS)                    # share of an onset day's cases already reported after L days
pm_full <- function(n) { pm <- diff(c(0, cc(0:(n - 1)))); M <- matrix(0, n, n); for (u in 1:n) M[u, 1:u] <- pm[u:1]; M }
```

```r
PM <- pm_full(TEND)

## predictions on the observation scale
predA <- function(inc) inc[dA$k] * cc(dA$lag)
predB <- function(inc) { Cc <- c(0, cumsum(as.vector(PM %*% inc))); Cc[dB$u + 1] - Cc[dB$u0 + 1] }

## piecewise-constant transmission: beta1 until day tc, beta2 afterwards; returns daily incidence on days 1..TEND
sim_fit <- function(i0, b1, b2, tc, eps = CFG$eps_base, sigma = CFG$sigma0, gamma = CFG$gamma0, tend = TEND) {
  rhs <- make_rhs(1, sigma, gamma, CFG$pop); y0 <- init_state(b1, sigma, gamma, CFG$tau0, eps, i0, 1)
  o1 <- ode(y0, 0:min(tc, tend), rhs, c(b1, CFG$tau0, eps, CFG$tau0, 0), method = "lsoda", rtol = 1e-6, atol = 1e-6)
  C <- o1[, 7]
  if (tc < tend) {
    o2 <- ode(o1[nrow(o1), -1], tc:tend, rhs, c(b2, CFG$tau0, eps, CFG$tau0, 0), method = "lsoda", rtol = 1e-6, atol = 1e-6)
    C <- c(C, o2[-1, 7])
  }
  diff(C)
}
nll_gen <- function(p4, tc, pred, y, ...) {                 # p4 = log i0, log beta1, log beta2, log k
  inc <- tryCatch(sim_fit(exp(p4[1]), exp(p4[2]), exp(p4[3]), tc, ...), error = function(e) NULL)
  if (is.null(inc) || any(!is.finite(inc))) return(1e10)
  -sum(dnbinom(y, mu = pmax(pred(inc), 1e-8), size = exp(p4[4]), log = TRUE))
}
nm2 <- function(f, start, it = 4000) { o <- optim(start, f, control = list(maxit = it, reltol = 1e-10)); optim(o$par, f, control = list(maxit = it, reltol = 1e-12)) }
fit_const  <- function(pred, y, start, ...) { f <- function(p) nll_gen(c(p[1], p[2], p[2], p[3]), TEND, pred, y, ...); o <- nm2(f, start)
  list(par4 = c(o$par[1], o$par[2], o$par[2], o$par[3]), nll = o$value, hess = optimHess(o$par, f), tc = TEND) }
fit_change <- function(pred, y, tc, start4, ...) { f <- function(p) nll_gen(p, tc, pred, y, ...); o <- nm2(f, start4)
  list(par4 = o$par, nll = o$value, hess = optimHess(o$par, f), tc = tc) }
mvn_draw <- function(mean, hess) {
  cv <- tryCatch(solve(hess), error = function(e) diag(1 / pmax(diag(hess), 1e-6))); cv <- (cv + t(cv)) / 2
  e <- eigen(cv); as.vector(mean + e$vectors %*% (sqrt(pmax(e$values, 1e-8)) * rnorm(length(mean))))
}
gr <- function(b, eps = CFG$eps_base, sigma = CFG$sigma0, gamma = CFG$gamma0) growth_rate(b, sigma, gamma, CFG$tau0, eps, 1)

analyse <- function(label, pred, y, start_const) {
  cat("\n==== ", label, " ====\n", sep = "")
  c0 <- fit_const(pred, y, start_const)
  grid <- as.numeric(seq(as.Date("2026-06-18"), as.Date("2026-08-28"), by = 5) - CFG$t0)
  prof <- list(); for (tc in grid) prof[[as.character(tc)]] <- fit_change(pred, y, tc, c0$par4)
  best <- as.numeric(names(prof))[which.min(sapply(prof, `[[`, "nll"))]
  for (tc in setdiff((best - 4):(best + 4), grid)) prof[[as.character(tc)]] <- fit_change(pred, y, tc, prof[[as.character(best)]]$par4)
  prof <- prof[order(as.numeric(names(prof)))]
  nll_tc <- sapply(prof, `[[`, "nll"); tcs <- as.numeric(names(prof)); best <- tcs[which.min(nll_tc)]
  b <- prof[[as.character(best)]]; ok <- tcs[nll_tc - min(nll_tc) <= 1.92]
  aic0 <- 2 * c0$nll + 6; aic1 <- 2 * b$nll + 10
  cat(sprintf("constant: nll %.2f r %.4f | changepoint %s: nll %.2f r %.4f -> %.4f | dAIC %.1f | tc 95%% %s to %s\n", c0$nll,
              gr(exp(c0$par4[2])), format(CFG$t0 + best), b$nll, gr(exp(b$par4[2])), gr(exp(b$par4[3])), aic0 - aic1,
              format(CFG$t0 + min(ok)), format(CFG$t0 + max(ok))))
  list(const = c0, prof = prof, tcs = tcs, nll_tc = nll_tc, best = b, aic0 = aic0, aic1 = aic1, ok = ok)
}
A <- analyse("Fit A: onset curve (reporting-delay corrected)", predA, dA$confirmed_total, c(log(25), log(0.245), log(20)))
B <- analyse("Fit B: confirmed cases by report date", predB, dB$y, c(log(35), log(0.245), log(30)))

## ---- uncertainty draws: change date ~ profile-likelihood weights; other parameters ~ MVN(Hessian) at that change date
draws_for <- function(F, pred, fit_label, nd = 1000, nband = 200) {
  w <- exp(-(F$nll_tc - min(F$nll_tc))); w <- w / sum(w)
  out <- vector("list", nd); bands <- list()
  for (i in 1:nd) {
    tc <- sample(F$tcs, 1, prob = w); fit <- F$prof[[as.character(tc)]]
    p4 <- mvn_draw(fit$par4, fit$hess); p4[2:3] <- pmin(pmax(p4[2:3], log(0.05)), log(1.5))
    inc <- tryCatch(sim_fit(exp(p4[1]), exp(p4[2]), exp(p4[3]), tc), error = function(e) NULL)
    if (is.null(inc)) next
    out[[i]] <- data.frame(fit = fit_label, tc_day = tc, logi0 = p4[1], beta1 = exp(p4[2]), beta2 = exp(p4[3]), k = exp(p4[4]),
      r1 = gr(exp(p4[2])), r2 = gr(exp(p4[3])), Re1 = Re_value(exp(p4[2]), CFG$gamma0, CFG$tau0, CFG$eps_base),
      Re2 = Re_value(exp(p4[3]), CFG$gamma0, CFG$tau0, CFG$eps_base), inc_end = inc[TEND])
    if (length(bands) < nband) bands[[length(bands) + 1]] <- list(inc = inc, mu = pred(inc))
  }
  list(draws = do.call(rbind, out), bands = bands)
}
cat("\nSampling parameter uncertainty ...\n")
DA <- draws_for(A, predA, "A"); DB <- draws_for(B, predB, "B")
write.csv(DA$draws, file.path(P$res, "fit_draws_A.csv"), row.names = FALSE); write.csv(DB$draws, file.path(P$res, "fit_draws_B.csv"), row.names = FALSE)

## ---- adequacy: 95% negative-binomial predictive coverage and residual autocorrelation (at the best fit)
adequacy <- function(F, pred, y) {
  b <- F$best; inc <- sim_fit(exp(b$par4[1]), exp(b$par4[2]), exp(b$par4[3]), b$tc); mu <- pred(inc); k <- exp(b$par4[4])
  lo <- qnbinom(.025, mu = mu, size = k); hi <- qnbinom(.975, mu = mu, size = k)
  pr_res <- (y - mu) / sqrt(mu + mu^2 / k)
  c(coverage95 = mean(y >= lo & y <= hi), resid_lag1_acf = acf(pr_res, plot = FALSE, lag.max = 1)$acf[2], resid_sd = sd(pr_res), k = k)
}
adq <- rbind(A = adequacy(A, predA, dA$confirmed_total), B = adequacy(B, predB, dB$y)); print(round(adq, 3))

## ---- sensitivity of the fitted growth rate to the fixed natural-history values (Fit B, change date fixed at best)
nh <- expand.grid(latent = c(4.0, 10.1), infectious = c(5, 10)); nh_rows <- list()
for (j in 1:nrow(nh)) {
  s <- 1 / nh$latent[j]; g <- 1 / nh$infectious[j]
  f <- function(p) nll_gen(p, B$best$tc, predB, dB$y, sigma = s, gamma = g)
  o <- nm2(f, B$best$par4, 2500)
  nh_rows[[j]] <- data.frame(latent = nh$latent[j], infectious = nh$infectious[j], nll = o$value,
    beta2 = exp(o$par[3]), r_after = growth_rate(exp(o$par[3]), s, g, CFG$tau0, CFG$eps_base, 1), r_before = growth_rate(exp(o$par[2]), s, g, CFG$tau0, CFG$eps_base, 1),
    inc_end = sim_fit(exp(o$par[1]), exp(o$par[2]), exp(o$par[3]), B$best$tc, sigma = s, gamma = g)[TEND])
}
nhtab <- do.call(rbind, nh_rows); print(round(nhtab, 4))
write.csv(nhtab, file.path(P$tab, "fit_naturalhistory_sensitivity.csv"), row.names = FALSE)
```

```r

## ---- beta at each isolation effectiveness (refit at the best change date)
eps_rows <- list()
for (e in CFG$eps) for (nm in c("A", "B")) {
  F <- if (nm == "A") A else B; pred <- if (nm == "A") predA else predB; y <- if (nm == "A") dA$confirmed_total else dB$y
  fit <- fit_change(pred, y, F$best$tc, F$best$par4, eps = e)
  eps_rows[[length(eps_rows) + 1]] <- data.frame(fit = nm, eps = e, nll = fit$nll, beta_before = exp(fit$par4[2]), beta_after = exp(fit$par4[3]),
    r_before = gr(exp(fit$par4[2]), e), r_after = gr(exp(fit$par4[3]), e),
    Re_before = Re_value(exp(fit$par4[2]), CFG$gamma0, CFG$tau0, e), Re_after = Re_value(exp(fit$par4[3]), CFG$gamma0, CFG$tau0, e))
}
epstab <- do.call(rbind, eps_rows); write.csv(epstab, file.path(P$tab, "fit_by_effectiveness.csv"), row.names = FALSE); print(round(epstab[, -1], 4))

saveRDS(list(A = A, B = B, DA = DA, DB = DB, adq = adq, nhtab = nhtab, epstab = epstab, TEND = TEND), file.path(P$res, "fit.rds"))
cat(sprintf("\nFit finished in %.1f min\n", as.numeric(difftime(Sys.time(), t_start, units = "mins"))))
```

C Code: shared model functions (R/00_functions.R)

```r
## 00_functions.R -- shared model code, helpers and constants (sourced by every script)
## Model: S-E-I-Q-R with an isolated compartment Q. The infectious (not yet isolated) class is split into
## m Erlang stages (m = 1 gives the plain exponential delay used in the main analysis; m > 1 is a
## structural sensitivity analysis). Isolation occurs on leaving stage m at rate m*tau (tau = 1/mean delay).
suppressPackageStartupMessages(library(deSolve))

## ------------------------------------------------------------------ paths
find_root <- function() {
  a <- commandArgs(FALSE); f <- sub("--file=", "", a[grep("--file=", a)])
  if (length(f)) normalizePath(file.path(dirname(f[1]), "..")) else normalizePath(".")
}
ROOT <- find_root()
P <- list(data = file.path(ROOT, "data"), bvd = file.path(ROOT, "data", "bvd_2026"),
          res = file.path(ROOT, "results"), fig = file.path(ROOT, "figures"),
          tab = file.path(ROOT, "results", "tables"))
for (d in c(P$res, P$fig, P$tab)) dir.create(d, FALSE, TRUE)

## ------------------------------------------------------------------ constants (single source of truth)
CFG <- list(
  sigma0 = 1 / 6.3,       # central latent period 6.3 d            (Ebengo, Akilimali et al. 2026 model input)
  gamma0 = 1 / 7,         # central infectious period 7 d          (idem; 6.5 d in Chowell & Nishiura 2014)
  lat_tri = c(4.0, 6.3, 10.1),   # triangular (min, mode, max) latent period, days
  inf_tri = c(5.0, 7.0, 10.0),   # triangular (min, mode, max) infectious period, days
  tau0 = 1 / 7,           # baseline onset-to-isolation delay 7 d (study brief)
  delays = c(7, 5, 3),    # scenario delays, days
  eps = c(0.5, 0.8, 1.0), # isolation effectiveness: worst / base / best
  eps_base = 0.8,
  pop = 4392200,          # Ituri population (scalars file); depletion negligible
  t0 = as.Date("2026-05-02"),      # model start for the fit
  cutoff = as.Date("2026-09-12"),  # last day of the surveillance data = start of scenarios
  horizon = 365,          # 12-month horizon for cumulative cases (days)
  max_days = 1825,        # 5-year window for outbreak duration
  cfr = 0.483             # 3,475 / 7,200 confirmed deaths / cases at 12 Sep (upper bound; README caveat 7)
)

## ------------------------------------------------------------------ small helpers
rtri <- function(n, p) {                    # triangular(min, mode, max) by inverse CDF
  u <- runif(n); a <- p[1]; c <- p[2]; b <- p[3]; Fc <- (c - a) / (b - a)
  ifelse(u < Fc, a + sqrt(u * (b - a) * (c - a)), b - sqrt((1 - u) * (b - a) * (b - c)))
}
q3 <- function(x, na.rm = TRUE) quantile(x, c(.5, .025, .975), na.rm = na.rm, names = FALSE, type = 1)
fmt <- function(x, d = 0, big = TRUE) formatC(x, format = "f", digits = d, big.mark = if (big) "," else "")
mci <- function(x, d = 0, big = TRUE) { q <- q3(x); sprintf("%s (%s--%s)", fmt(q[1], d, big), fmt(q[2], d, big), fmt(q[3], d, big)) }
tex_escape_pct <- function(s) gsub("%", "\\\\%", s)

## ------------------------------------------------------------------ deterministic model
## NB: layout of the state vector for the Jacobian is (E, I_1..I_m, Q); S, R and the case counter are not needed.
jac_m <- function(beta, sigma, gamma, tau, eps, m = 1) {
  n <- m + 2; J <- matrix(0, n, n); r <- m * tau
  J[1, 1] <- -sigma; J[1, 2:(1 + m)] <- beta; J[1, n] <- beta * (1 - eps)
  J[2, 1] <- sigma
  for (k in 1:m) { J[1 + k, 1 + k] <- -(r + gamma); if (k > 1) J[1 + k, k] <- r }
  J[n, 1 + m] <- r; J[n, n] <- -gamma
  J
}
growth_rate <- function(beta, sigma, gamma, tau, eps, m = 1) max(Re(eigen(jac_m(beta, sigma, gamma, tau, eps, m), only.values = TRUE)$values))
## Reproduction number at the disease-free state: R = (beta/gamma) * (1 - eps * a^m),  a = m*tau/(m*tau + gamma)
Re_value <- function(beta, gamma, tau, eps, m = 1) { a <- m * tau / (m * tau + gamma); (beta / gamma) * (1 - eps * a^m) }
frac_isolated <- function(gamma, tau, m = 1) (m * tau / (m * tau + gamma))^m       # share of cases isolated before recovery/death

calibrate_beta <- function(r, sigma, gamma, tau0, eps, m = 1)
  uniroot(function(b) growth_rate(b, sigma, gamma, tau0, eps, m) - r, c(1e-4, 6), tol = 1e-12)$root

make_rhs <- function(m, sigma, gamma, pop) {
  function(t, y, p) {                       # p = c(beta, tau_new, eps, tau0, ramp_days)
    beta <- p[1]; eps <- p[3]
    tau <- if (p[5] > 0) p[4] + (p[2] - p[4]) * min(1, t / p[5]) else p[2]
    S <- y[1]; E <- y[2]; I <- y[3:(2 + m)]; Q <- y[3 + m]
    lam <- beta * (sum(I) + (1 - eps) * Q) / pop
    rate <- m * tau
    dI <- c(sigma * E, rate * I[-m]) - (rate + gamma) * I
```

```
    list(c(-lam * S, lam * S - sigma * E, dI, rate * I[m] - gamma * Q, gamma * (sum(I) + Q), sigma * E))
  }
}
## Initial state: dominant eigenvector of the linearised (E, I_1..I_m, Q) system at the BASELINE delay, scaled so that sigma*E = i0
init_state <- function(beta, sigma, gamma, tau0, eps, i0, m = 1, pop = CFG$pop) {
  ev <- eigen(jac_m(beta, sigma, gamma, tau0, eps, m)); v <- abs(Re(ev$vectors[, which.max(Re(ev$values))]))
  v <- v * i0 / (sigma * v[1])
  c(S = pop - sum(v), E = v[1], I = v[2:(1 + m)], Q = v[2 + m], R = 0, C = 0)
}
## Run one scenario. Returns daily symptomatic incidence (length tend) and active infections (E+I+Q) at each day.
run_det <- function(beta, sigma, gamma, tau_new, eps, i0, tau0 = CFG$tau0, m = 1, tend = CFG$max_days,
                    ramp = 0, pop = CFG$pop, y0 = NULL) {
  if (is.null(y0)) y0 <- init_state(beta, sigma, gamma, tau0, eps, i0, m, pop)
  o <- ode(y0, 0:tend, make_rhs(m, sigma, gamma, pop), c(beta, tau_new, eps, tau0, ramp),
           method = "lsoda", rtol = 1e-7, atol = 1e-7)
  nm <- m + 5
  list(inc = diff(o[, nm + 1]), active = rowSums(o[, 3:(3 + m + 1)]) , cum = o[, nm + 1])
}
## active infections = E + sum(I_k) + Q = columns 3 .. (3+m+1) of the ode matrix (col 1 = time, col 2 = S)
outcomes <- function(sim, horizon = CFG$horizon) {
  ext <- which(sim$active < 1)[1]                      # first day with < 1 exposed/infectious person
  list(cum = sim$cum[horizon + 1] - sim$cum[1], duration = if (is.na(ext)) Inf else ext - 1,
       peak = max(sim$inc[1:horizon]), tpeak = which.max(sim$inc[1:horizon]))
}

## ------------------------------------------------------------------ discrete-time stochastic model (m = 1)
disc_matrix <- function(beta, sigma, gamma, tau, eps, dt) {
  es <- exp(-sigma * dt); et <- exp(-tau * dt); eg <- exp(-gamma * dt)
  matrix(c(es, beta * dt, beta * dt * (1 - eps), 1 - es, et * eg, 0, 0, 1 - et, eg), 3, 3, byrow = TRUE)
}
calibrate_beta_disc <- function(r, sigma, gamma, tau0, eps, dt)
  uniroot(function(b) max(Re(eigen(disc_matrix(b, sigma, gamma, tau0, eps, dt), only.values = TRUE)$values)) - exp(r * dt),
          c(1e-4, 6), tol = 1e-12)$root
init_disc <- function(beta, sigma, gamma, tau0, eps, i0, dt, pop = CFG$pop) {
  ev <- eigen(disc_matrix(beta, sigma, gamma, tau0, eps, dt)); v <- abs(Re(ev$vectors[, which.max(Re(ev$values))]))
  v * i0 * dt / ((1 - exp(-sigma * dt)) * v[1])
}
## Vectorised over replicates: every argument may be a vector of length nrep. Binomial-chain (tau-leap) transitions.
run_stoch <- function(beta, sigma, gamma, tau, eps, E0, I0, Q0, tend = CFG$max_days, dt = 0.25, pop = CFG$pop,
                      horizon = CFG$horizon) {
  n <- length(beta); S <- rep(round(pop), n); E <- round(E0); I <- round(I0); Q <- round(Q0)
  C12 <- numeric(n); ext <- rep(NA_real_, n); steps <- as.integer(tend / dt)
  pE <- 1 - exp(-sigma * dt); pQ <- 1 - exp(-tau * dt); pG <- 1 - exp(-gamma * dt)
  for (k in seq_len(steps)) {
    lam <- beta * (I + (1 - eps) * Q) / pop
    nE <- rbinom(n, S, 1 - exp(-lam * dt)); nI <- rbinom(n, E, pE)
    nQ <- rbinom(n, I, pQ); rI <- rbinom(n, I - nQ, pG); rQ <- rbinom(n, Q, pG)
    S <- S - nE; E <- E + nE - nI; I <- I + nI - nQ - rI; Q <- Q + nQ - rQ
    if (k * dt <= horizon) C12 <- C12 + nI
    done <- is.na(ext) & (E + I + Q == 0); ext[done] <- k * dt
  }
  list(cum12 = C12, ext = ext)
}

## ------------------------------------------------------------------ writing numbers for LaTeX
MACROS <- new.env()
macro_reset <- function() rm(list = ls(MACROS), envir = MACROS)
macro <- function(name, value) assign(name, as.character(value), envir = MACROS)
macro_write <- function(file) {
  nm <- sort(ls(MACROS)); lines <- vapply(nm, function(n) sprintf("\\newcommand{\\%s}{%s}", n, get(n, MACROS)), "")
  writeLines(c("% AUTO-GENERATED by R/07_figures_tables.R -- do not edit", lines), file)
}
```

References

[1] Sam Abbott, Katharine Sherratt, et al. BVDOutbreakSize: analysis and cleaned surveillance streams for the 2026 Bundibugyo virus disease outbreak. https://github.com/epiforecasts/BVDOutbreakSize, 2026. Data as of 12 September 2026; underlying sources: INSP situation reports and INRB-UMIE transcriptions.